

Prompt: Act as a Windows Ansible expert. I want to remote into a Windows computer as an administrator to install software from Linux or WSL using Ansible.

Create an Ansible playbook and inventory file that will open a command prompt as an administrator, and run **winget** to install these programs:

- Adobe Acrobat Reader
- VS Code
- Notepad++
- Microsoft Office 365
- Firefox
- Chrome
- IObit Advanced System Care Ultimate
- Cisco Packet Tracer
- Cisco Webex
- WireShark
- VMware Workstation Pro

Then, turn on Windows Subsystem for Linux with the latest version of **Ubuntu** and **Virtual Machine Platform**.

Finally, run **winget upgrade -all**

Windows Software Installation Automation with Ansible

Key Components

- Inventory file configuration
- Ansible playbook implementation
- WinRM setup instructions
- Complete installation workflow

Prerequisites

- Windows Target Machine Requirements
 - Windows 10/11
 - PowerShell 5.1 or newer
 - .NET Framework 4.0 or newer
 - Administrator access
 - WinRM enabled and configured
- Linux/WSL Control Machine Requirements
 - Python 3.x
 - Ansible (latest version)
 - Required Python packages installed

Instructions

Task 1: Configure the Windows target machine with Windows Remote Management (WinRM)

Note: When using automation tools like Ansible, having WinRM configured on the target Windows machines is crucial. Ansible uses WinRM to communicate with Windows hosts, allowing it to execute tasks, manage configurations, and deploy applications remotely. Without WinRM, Ansible would not be able to manage Windows systems effectively.

Step 1: Enable WinRM: Run the following PowerShell commands as Administrator on the Windows machine to enable WinRM:

```
winrm quickconfig
Set-Item WSMan:\localhost\Service\Auth\Basic $true
Set-Item WSMan:\localhost\Service\AllowUnencrypted True
netsh advfirewall firewall add rule name="WinRM HTTP" dir=in action=allow
protocol=TCP localport=5985, 5986
winrm get winrm/config/service
```

- a. Verify WinRM Configuration: Run the following command from a command prompt to check the WinRM listener configuration:

```
winrm e winrm/config/listener
```

Ensure that you see listeners for both HTTP (port 5985) and HTTPS (port 5986) in the output.

- b. Test Connectivity: From a remote machine, you can test the connection to the WinRM service using:

```
nc -vz <Windows_IP_Address> 5985
nc -vz <Windows_IP_Address> 5986
```

Replace <Windows_IP_Address> with the actual IP address of your Windows machine.

- c. Test WinRM Connectivity to ensure that remote connections can be established successfully using the following command:

```
winrm id -uri http://YourHostname:5985/wsman
```

Replace *YourHostname* with your actual hostname. This command should return information about the WinRM service if everything is configured correctly.

Task 2: Install Ansible, pywinrm, and pypsrp on the control node

Note: You can use WSL, a virtual machine, or real Linux computer as the control node.

```
sudo apt update
sudo apt install python3 python3-pip ansible-core -y
pip3 install ansible pywinrm pypsrp --break-system-packages
ansible --version
```

Task 3: Create the working directory and Initial Files

Step 1: In your Home directory, create a directory named **bin**:

```
cisco@SCOTT_PC:~$ mkdir bin
```

Step 2: Shutdown WSL, wait 10 seconds, and restart.

Step 3: Navigate to the bin directory:

```
cisco@SCOTT_PC:~$ cd bin
```

Step 4: Create another directory named **setup_windows**:

```
cisco@SCOTT_PC:~/bin$ mkdir setup_windows
```

Step 5: Navigate into the new directory:

```
cisco@SCOTT_PC:~/bin$ cd setup_windows
```

Step 6: Create the need files:

```
cisco@SCOTT_PC:~/bin/setup_windows$ touch ansible.cfg con_test.yml
inventory.yml win_test.yml windows_setup.yml windows_update.yml
```

Step 7: Check the file structure:

```
cisco@SCOTT_PC:~/bin/setup_windows$ ls
ansible.cfg con_test.yml inventory.yml win_test.yml windows_setup.yml
windows_update.yml
```

Task 4: Create an Ansible Configuration File

Step 1: Open the **ansible.cfg** configuration file and add this information:

```
cisco@SCOTT_PC:~/bin/setup_windows$ nano ansible.cfg

[defaults]
inventory = ./inventory.yml
remote_user = YOUR_ADMIN_USER # Replace with the administrator username
host_key_checking = False

[privilege_escalation]
become = True
become_method = runas

[winrm_connection]
scheme = http
```

Step 2: Save and close (x,y,enter).

Task 5: Create Ansible Inventory Files

Step 1: Open the `inventory.yml` file and add this information

```
cisco@SCOTT_PC:~/bin/setup_windows$ nano inventory.yml

---
windows:
  hosts:
    win_host:
      ansible_host: 192.168.1.100 # Replace with the IP address of the
Windows machine
      ansible_user: Administrator # Replace with the administrator
username
      ansible_password: YourPasswordHere # Replace with the
administrator password
      ansible_connection: winrm
      ansible_winrm_server_cert_validation: ignore
      ansible_port: 5985
  ...
```

Step 2: Save and close (x,y,enter).

Task 6: Create a Basic Connection Test Playbook.

Step 1: Create a simple playbook to test connectivity and privilege escalation:

```
---
- name: Test Windows Connection
  hosts: windows
  gather_facts: false
  become: true
  become_user: Administrator # Specify the user to run as

  tasks:
    - name: Test connection
      ansible.windows.win_ping:
  ...
```

Step 2: Save and close (x,y,enter).

Step 3: Run the Playbook from a Linux Bash terminal:

```
ansible-playbook -i inventory.yml con_test.yml
```

Task 7: Test the connection between the control node and the remote system

The following playbook demonstrates how to connect to a Windows machine as an administrator and execute a command (e.g., opening a command prompt or running a script).

Step 1: Open the `win_test.yml` file and add this information:

```
cisco@SCOTT_PC:~/bin/setup_windows$ win_test.yml
---
- name: Remote into Windows and execute commands as Administrator
  hosts: windows
  gather_facts: true
```

```

become: true
become_user: Administrator # Specify the user to run as

pre_tasks:
  - name: Ensure Windows Update service is running
    ansible.windows.win_service:
      name: wuauserv
      state: started
      register: wuauserv_status
      failed_when: false # Do not fail if the service is already
running
      changed_when: wuauserv_status.state != "running" # Mark as
changed only if the service was not running

  - name: Debug Windows Update service status
    ansible.builtin.debug:
      msg: "Windows Update service is {{ wuauserv_status.state }}"

tasks:
  # Verify connection to the Windows host
  - name: Verify connection to the Windows host
    win_ping:

  # Run a command as Administrator
  - name: Open Command Prompt as Administrator
    win_shell: |
      Start-Process cmd.exe -Verb runAs
    register: cmd_output

  # Display the output of the command
  - name: Display command output
    debug:
      var: cmd_output.stdout

  # Example: Run a custom PowerShell command as Administrator
  - name: Run a PowerShell command
    win_shell: |
      Write-Output "Hello from Ansible as Administrator"
    register: ps_output

  # Display the output of the PowerShell command
  - name: Display PowerShell output
    debug:
      var: ps_output.stdout
...

```

Step 2: Save and close (x,y,enter).

Step 3: Run the Playbook from a Linux Bash terminal:

```
ansible-playbook -i inventory.yml win_test.yml
```

Task 8: Create a windows_setup.yml Ansible Playbook

Step 1: Open a new document named windows_setup.yml and copy this code into it:

```
cisco@SCOTT_PC:~/bin/setup_windows$ nano windows_setup.yml
---
- name: Install software and configure Windows features
  hosts: windows
  gather_facts: true
  become: true
  become_user: Administrator # Specify the user to run as

  pre_tasks:
    - name: Ensure Windows Update service is running
      ansible.windows.win_service:
        name: wuauserv
        state: started
        register: wuauserv_status
        failed_when: false # Do not fail if the service is already
running
        changed_when: wuauserv_status.state != "running" # Mark as
changed only if the service was not running

    - name: Debug Windows Update service status
      ansible.builtin.debug:
        msg: "Windows Update service is {{ wuauserv_status.state }}"

    - name: Debug Windows Update service status
      ansible.builtin.debug:
        msg: "Windows Update service is {{ wuauserv_status.state }}"

  tasks:
    # Verify Administrative Privileges
    - name: Check for administrative privileges
      win_shell: net session
      register: admin_check
      ignore_errors: true

    - name: Fail if not running as administrator
      fail:
        msg: "This playbook requires administrative privileges"
        when: admin_check.rc != 0

    # Software Installation
    - name: Install required software packages
      block:
        - name: Install Adobe Acrobat Reader
          win_shell: winget install --id Adobe.Acrobat.Reader.64-bit --
silent
          register: acrobat_install

        - name: Install Visual Studio Code
```

```

        win_shell: winget install --id Microsoft.VisualStudioCode --
silent      register: vscode_install

    - name: Install Notepad++
      win_shell: winget install --id Notepad++.Notepad++ --silent --
accept-package-agreements --accept-source-agreements
      register: notepad_install

    - name: Install Microsoft Office 365
      win_shell: winget install --id Microsoft.Office --silent
      register: office_install

    - name: Install Firefox
      win_shell: winget install --id Mozilla.Firefox --silent
      register: firefox_install

    - name: Install Google Chrome
      win_shell: winget install --id Google.Chrome --silent
      register: chrome_install

    - name: Install IObit Advanced SystemCare Ultimate
      win_shell: winget install --id
IObit.AdvancedSystemCareUltimate --silent --accept-package-agreements --
accept-source-agreements
      register: iobit_install

    - name: Install Cisco Packet Tracer
      win_shell: winget install --id Cisco.PacketTracer --silent
      register: packettracer_install

    - name: Install Cisco Webex
      win_shell: winget install --id Cisco.Webex --silent
      register: webex_install

    - name: Install WireShark
      win_shell: winget install -e --id
WiresharkFoundation.Wireshark --silent
      register: wireshark_install

    - name: Install VMware Workstation Pro
      win_shell: winget install --id VMware.WorkstationPro --silent
      register: vmware_install
  rescue:
    - name: Log installation failure
      win_shell: Write-EventLog -LogName Application -Source
"Ansible" -EventId 1000 -EntryType Error -Message "Software installation
failed"

# WSL and Virtual Machine Platform Setup
- name: Enable Windows Features

```

```

    block:
      - name: Enable Windows Subsystem for Linux
        win_feature:
          name: Microsoft-Windows-Subsystem-Linux
          state: present
          register: wsl_install

      - name: Enable Virtual Machine Platform
        win_feature:
          name: VirtualMachinePlatform
          state: present
          register: vmp_install

      - name: Restart Windows if required
        win_reboot:
          when: wsl_install.reboot_required or
vmp_install.reboot_required
        rescue:
          - name: Log Windows Features failure
            win_shell: Write-EventLog -LogName Application -Source
"Ansible" -EventId 1001 -EntryType Error -Message "Failed to enable
Windows Features"

# WSL2 Configuration
- name: Configure WSL2
  block:
    - name: Set WSL 2 as default version
      win_shell: wsl --set-default-version 2
      register: wsl2_default

    - name: Install Ubuntu
      win_shell: wsl --install -d Ubuntu
      register: ubuntu_install
  rescue:
    - name: Log WSL configuration failure
      win_shell: Write-EventLog -LogName Application -Source
"Ansible" -EventId 1002 -EntryType Error -Message "WSL configuration
failed"

# Package Updates
- name: Update all installed packages
  block:
    - name: Run winget upgrade
      win_shell: winget upgrade --all --silent
      register: upgrade_result
  rescue:
    - name: Log upgrade failure
      win_shell: Write-EventLog -LogName Application -Source
"Ansible" -EventId 1003 -EntryType Error -Message "Package upgrade
failed"
...

```

Step 2: Save and close (x,y,enter).

Step 3: Run the Playbook from a Linux Bash terminal:

```
ansible-playbook -i inventory.yml windows_setup.yml -vvvv
```

Task 9: Create a window_update.yml Ansible Playbook

Step 1: Open a new document named `windows_update.yml` and copy this code into it:

```
cisco@SCOTT_PC:~/bin/setup_windows$ nano windows_update.yml
---
- name: Manage Windows Updates
  hosts: windows
  gather_facts: true
  become: true
  become_user: Administrator # Specify the user to run as

  pre_tasks:
    - name: Ensure Windows Update service is running
      ansible.windows.win_service:
        name: wuauserv
        state: started
        register: wuauserv_status
        failed_when: false # Do not fail if the service is already
running
        changed_when: wuauserv_status.state != "running" # Mark as
changed only if the service was not running

    - name: Debug Windows Update service status
      ansible.builtin.debug:
        msg: "Windows Update service is {{ wuauserv_status.state }}"

  tasks:
    - name: Install Windows updates
      block:
        - name: Search for available Windows updates
          ansible.windows.win_updates:
            state: searched
            register: update_search

        - name: Install all Windows updates if available
          ansible.windows.win_updates:
            category_names:
              - SecurityUpdates
              - CriticalUpdates
              - UpdateRollups
              - Updates
            state: installed
            reboot: true
            reboot_timeout: 3600
            log_path: C:\ansible_win_updates.log
            register: update_result
            when: update_search.found_update_count > 0
```

```

        - name: Verify Windows updates installation
          ansible.windows.win_shell: Get-HotFix | Sort-Object -Property
InstalledOn -Descending | Select-Object -First 1
          register: hotfix_check
      rescue:
        - name: Handle Windows Update failure
          ansible.builtin.fail:
            msg: "Failed to install Windows updates. Please check
C:\ansible_win_updates.log for details."

    post_tasks:
      - name: Final status report
        ansible.builtin.debug:
          msg: |
            Update Summary:
            Windows Updates: {{ "Successful" if
update_result.reboot_required is defined else "No updates required" }}
        ...

```

Step 2: Save and close (x,y,enter).

Step 3: Run the Playbook from a Linux Bash terminal:

```
ansible-playbook -i inventory.yml windows_update.yml -vvvv
```